Chapter 1

Sound Pressure variations can be expressed in the form of analog:

- Mechanical displacement (microphone, speaker)
- Voltage variations
- Wiggles in vinyl record grooves
- Degree of magnetization on tape
- Optical density in film

Digital → analog: seq of numbers to control voltage; filter the voltage to produce a smooth signal

Nyquist
`print expr`: evaluate expression and print result
`exec expr`: evaluate expression (no print)
`play expr`: evaluate and play the *sound*
`display var`: print the variable information for debugging
`set var = expr`: set a variable
Symbols can contain `* - +` etc. and are case-insensitive
Keyword parameters are optional and are matched by keyword rather than by position
`with` introduces local variables, default init `nil`
Use `begin-end` to contain nested `if-then-else` statments
Loop:

- `for i from 0 below 10`: $i = 0, \ldots, 9$
- `for elem in my-list`: iterate over list elements
- `for v = init then update`: flexible update
- `while expr` / `until expr`: arbitrary stop condition
- `repeat n`: iterate n times
- `finally return local-var`: executed once at the end

Chapter 2

Unit Generator
Primitives in a sound synthesis system that perform sound generation and sound processing Unit generators are functions on sounds
Lazy evaluation on sounds
Implemented as implicit *object* with the current phase and frequency (MAX MSP: explicit graph)

No direct access to UG but is contained in *sounds* as returned by functions as objects
`set var = sound-expr`: the entire sound is computed and stored in memory
Use function programming to wrap global variables

Wavetable Synthesis

```
define variable *table* =
  sim(0.5 * build-harmonic(1.0, 2048),
     0.25 * build-harmonic(2.0, 2048),
     0.125 * build-harmonic(3.0, 2048),
     0.062 * build-harmonic(4.0, 2048))
set *table* = list(*table*, hz-to-step(1), #t)
```

Piece-wise Linear Functions: `pwl(t1, v1, t2, v2, ..., tn)`, `pwlv(v0, t1, v1, t2, v2, ..., tn, vn)` (nonzero starting/ending points), `pwe` (exponential interpolation), `pwlr` (relative intervals)
Advantages: simple, efficient, direct control
Disadvantages: spectrum (strength of harmonics) does not change with pitch or time as in most acoustic instruments

Scores

Pitch: "flats" lower by half step, "sharps" raise by half step, whole step = 2 half steps, `c4 = 60`
Steps are logarithms of frequency, frequency doubles every 12 steps ⇒ frequency doubling = octave
Harmonics: a periodic function of time hearing as a tone with pitch, fundamental frequency = 1/period
Any continuous function can be decomposed into a sum of sinusoids
Periodic functions can be decomposed into sinusoids with frequencies that are integer multiples of the fundamental frequency
Score: {`sound-event sound-event ...`}
A sound-event is {`time duration sound`}

Chapter 3

Sampling Theory

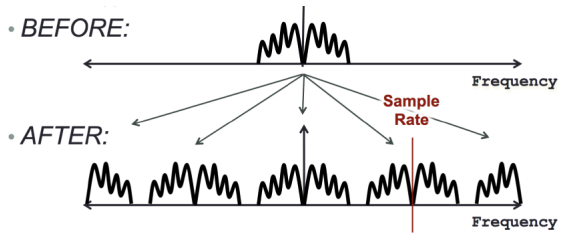
Sinusoidal partials: $A \cdot \sin(\omega t + \phi)$
Fourier transflorm (*not a function of time*) expresses the time function as an (infinite) sum of sinusoids and expresses the infinite sum as a function from frequency to amplitudes:

$$R(\omega) = \int_{-\infty}^{\infty} f(t) \cos \omega t dt, \qquad X(\omega) = \int_{-\infty}^{\infty} f(t) \sin \omega t dt$$
$$A(\omega) = \sqrt{R^2(\omega) + X^2(\omega)}, \qquad \phi(\omega) = \arctan(X(\omega)/R(\omega))$$
$$F(\omega) = R(\omega) + j \cdot X(\omega) = \int_{-\infty}^{\infty} f(t) e^{-j\omega t} dt$$

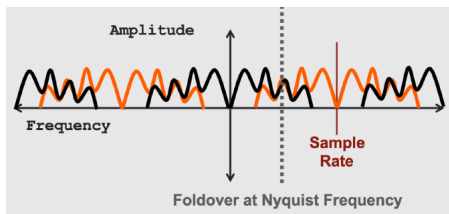
Every signal is a unique linear combination of sinusoids
Ignore phase: phase generally cannot be heard; amplitude spectrum is unchanged after Δt

Perfect Sampling

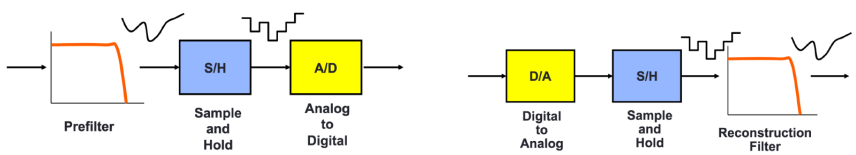
Multiplication in the time domain = convolution in the frequency domain
Convolution with a pulse train copies and shifts the spectrum



Aliasing: new frequencies are introduced and added to the original spectrum



The frequency range (bandwidth) of a sampled signal is determined by the sample rate
1/2 sampling rate: *Nyquist rate / frequency*



An impulse has infinite bandwidth
Bijection between samples and signal
Interpolation / Reconstruction: *convolve* with a sinc function (form the superposition of sinc functions shifted by the sample times and scaled by the sample values, requires infinite computation but good approximations are possible)

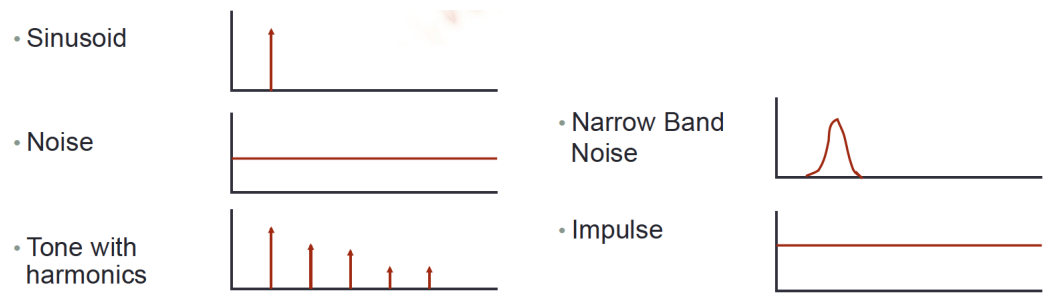
Imperfect Sampling

Signal to Noise measures *maximum* signal to noise
1dB $= 10\log_{10}\left(\frac{P_2}{P_1}\right)$, power $\propto$ amplitude^2
e.g. 10dB $= \times 10$ power, 20dB $= \times 100$ power $= \times 10$ amplitude
Quantization noise: signal rounded to the nearest integer sample
True signal = rounded signal + quantization error (uniformly randomly distributed in $[-0.5, +0.5]$)
Signal-to-Noise Ratio(SNR, dB) $= 6.02M + 1.76$ given M-bit sample
The signal-to-noise ratio is determined by the bits per sample
Discrete samples can capture a continuous band-limited signal without loss

Dither and Oversampling

Dither: add analog noise prior to quantization to decorrelate rounding error to signal
Oversampling: digitally reconstruct signal at high sample rate $\Rightarrow$ simpler to build analog filter

The Frequency Domain



Negative frequencies contain redundant information and are usually ignored
Short-term FT (STFT): $F(\omega) = \int_a^b f(t)e^{-j\omega t}dt$
Discrete FT (DFT)
Fast FT (FFT): $F_k = \sum_{n=a}^{b} f_n e^{-j2\pi kn/N}$ $(O(n \log n))$
Windowing: reduce the artifacts of the STFT

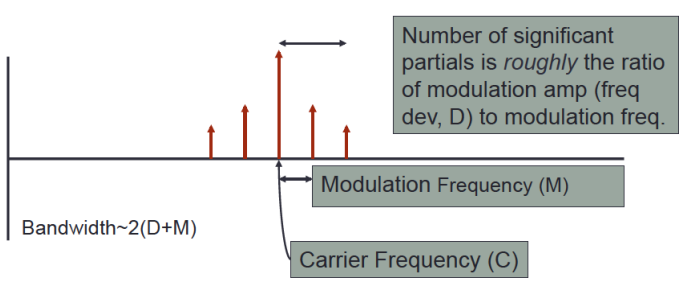
Amplitude Modulation

Amplitude modulation (multiplication) in the time domain = convolution in the spectral: for each sinusoid in the modulator, the modulated signal is shifted up/down by frequency of the sinusoid
Multiply two sinusoids together to obtain all of the sum and difference frequencies of all of the sinusoids in each signal
`lfo(vibrato-rate) * frequency-deviation`
Add a constant to `lfo` to avoid doubling frequency

Chapter 4

FM Modulation

`fmosc(basic-pitch, fm-control [, table [, phase]])`: deviation in Hz
`hzosc(fm-control)`: absolute frequency in Hz; e.g. `hzosc(abs-freq * pwlv(freq-control))`
`snd-compose(f, g)`: computes $f(g(t))$
FM synthesis provides time-varying control over the spectrum



Example: `fmosc(carrier-freq, mod-depth * osc(mod-freq))`
Depth of modulation: D, modulation frequency: M, carrier frequency: C
The number of sidebands above/below the carrier frequency is roughly equivalent to $I + 1$ where $I = D/M$
In theory, there are infinitely many sidebands at $C \pm kM, k \in \mathbb{N}$
The negative frequencies will sound like positive frequencies

Chapter 4

Behavioral Abstraction

The specific sound computed by the expression depends upon the *environment* (starting time, stretch factor, transposition, legato factor, loudness, sample rates, etc.)
Environment scoping is dynamic and can be overrided

The seq Construct

Evaluate the first sound at default time, resulting sound has a *logical stop time*; evaluate the second sound immediately after; return the sum of sounds

```
set x = osc(c4) ; compute sounds
set y = osc(d4) ;
play seq(x, y)  ; WRONG!!
```

```
function x() return osc(c4) ; define
function y() return osc(d4) ;  behaviors
play seq(x(), y())  ; RIGHT!!
```

The sim Construct

Evaluate a list of behaviors and forms their sum (equiv to audio mixing)

Logical Stop Time

The *signal stop* is when the sound samples end. After that point, the sound is considered to continue forever with value of zero. The *logical stop* marks the "musical" or "rhythmic" end of the sound. If there is a sound after it (e.g. in a sequence computed by seq), the next sound begins at this logical stop time of the sound

Chapter 5

Spectral Centroid

A control of brightness
Point $RN/2$ is at $SR/2$, Points X_k and R_k are at $(k/(N/2)) * (SR/2) = k * SR/N$ Hz, $X_0, X_{N/2}$ are always 0 and are thus omitted: $R_0, R_1, X_1, R_2, X_2, \ldots, R_{N/2}$
Frequency spacing (bin width) is N/SR Hz
FFT takes in N samples and output N values
Discrete magnitude spectrum: magnitude $A_k = \sqrt{R_k^2 + X_k^2}, k = 0, \ldots, N/2$
Weighted average of the amplitude spectrum (w is the width of each spectral bin in Hz = sample rate / size of the FFT in samples)

$$\text{spectral centroid} = \frac{\sum_{i=0}^{N} i \cdot w \cdot A_i}{\sum_{i=0}^{N} A_i}$$

Algorithmic Composition

`score-gen(attr: expr, attr: exp, ...)`
`sg:start`: starting time for current note
`sg:ioi`: current inter-onset interval (separation between starting times)
`sg:dur`: current duration
`sg:count`: number of notes computed so far

Patterns

`make-cycle(list)`
`make-heap(list-or-pattern, for: no-or-pat, max: no-or-pat)` outputs elements of input list in random order, `for` specifies length of period, `max: 1` forces no repeat of elements
`make-palindrome(list-or-pat, for: no-or-pat, elide: keyword)` outputs elements of input list forward then reverse order, `elide`:

- `first`: ABCCBABCCB
- `last`: ABCBAABCBA
- `#t`: ABCBABCB
- `#f`: ABCCBAABCCBA

`make-random(list-or-pat, for: no-or-pat)` selects items from list at random
`make-line(list-or-pat, for: no-or-pat)` outputs elements of list with last element repeat
`make-accumulation(list-or-pat, for: no-or-pat)` outputs initial substrings
Length of pattern period can be specified by `for`; default is length of input
`make-accumulate(pat, max: expr, min: expr, for: no-or-pat)` sums successive elements from input pattern
`make-copier(pat, repeat: expr-or-pat, merge: boolean, for: no-or-pat)` copies each period repeat times and merges to one period if `merge = #t`
`make-length(list-or-pat, no-or-pat)` regroups input sequence to specified period lengths
`make-window(pat, win-size, win-skip)` outputs `win-size` elements then advances `win-skip`

Nested Patterns

When a pattern generator accesses a list or other parameter and gets a pattern object, it uses the next value generated by the pattern object

Tendency Masks

Manual parametric change allows composition of global trends

Chapter 6

Recursive Sounds

Multiplying limited sound by an infinite recursive sound gives a finite computation and result

Matching Durations

1. Make everything have nominal length of 1 and use STRETCH;
2. Provide duration parameters everywhere.

Control Functions

`lfo` – low frequency sinusoid
`pwl` – arbitrary contours and shapes
`noise` – random jitter
Control functions from PWL, LFO, etc. can be passed as parameters and returned from functions;

they are of type SOUND

Granular Synthesis

Combine many grains of sound and apply smooth envelope to avoid clicks

---

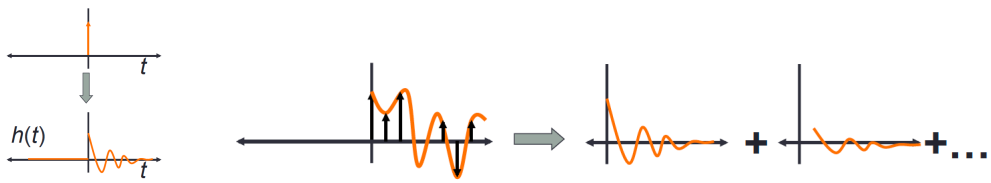
Chapter 7

Sampling Synthesis

Advantages: easily captured sound, works with noise, tones, etc.

Filters

Suppose we know what a filter does to an impulse, then the *impulse response* $h(t)$ of the filter:



Most filters are *linear time invariant*: Response(A+B) = Response(A) + Response(B)

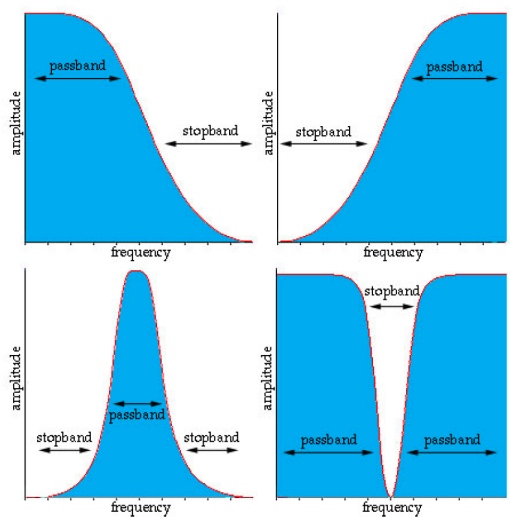
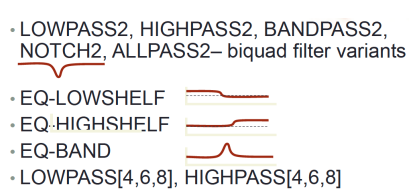


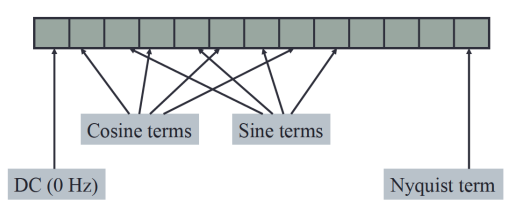
Figure 7.5: Four common filter types (clockwise from upper left): low-pass, high-pass, band-reject, band-pass.

A low-/high-pass filter removes frequencies above/below some *cutoff frequency*
A band-pass filter lets through only frequencies above a certain point and below another
A band-reject filter (*notch filter*) stops a band of frequencies



Spectral Processing

Representation: spectra appear as floating point arrays



Perfect reconstruction:

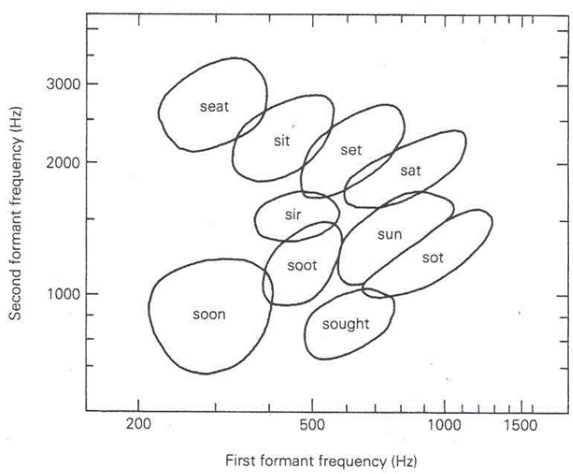
- FFT can be inverted: IFFT
- windows are applied twice: Window → FFT → (alter signal) → IFFT → Window
- A step size of 25% of the window duration (a 75% overlap) is required for perfect reconstruction

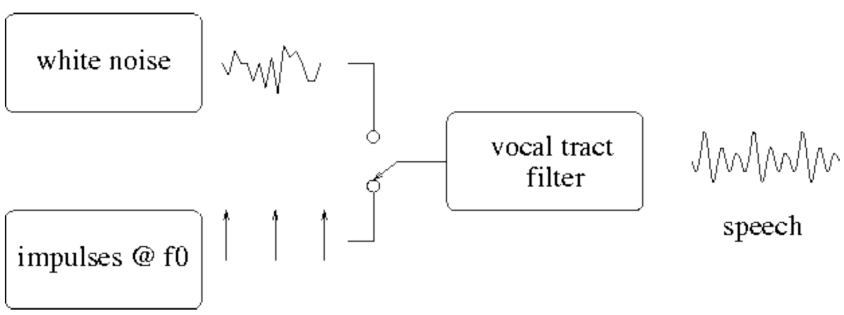
```
set sa = sa-init(input: "./rpd-cello.wav",
                 fft-dur: 4096 / 44100.0,
                 skip-period: 512 / 44100.0,
                 window: :hann)

set sp = sp-init(sa, quote(processing-fn), 0, 0)
function processing-fn(sa, frame, p1, p2)
    begin
        ... Process frame here ...
        set frame[0] = 0.0 ; simple example: remove DC
        return list(frame, f(p1), g(p2)) ; state change
    end

play sp-to-sound(sp)
```

Human Voice

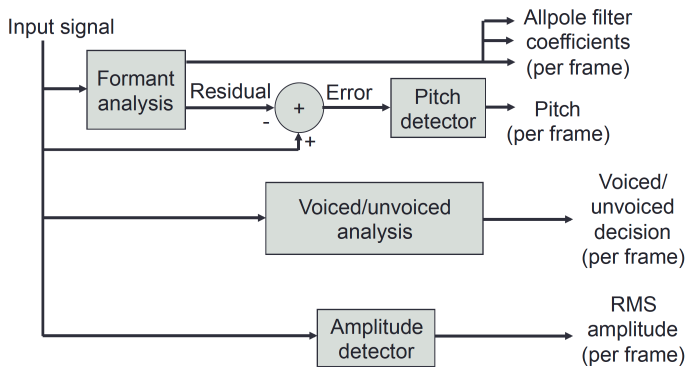




LPC

Linear Prediction Coding predicts next sample as a weighted sum of past samples: $s_n = \sum_{i=1}^{p} a_i s_{n-i} \Rightarrow$ the Vocal Tract Filter: *allpole* filter (the response consists of resonant peaks)

LPC creates an inverse filter ⇒ residual ⇒ an estimate of glottal pulses ⇒ do pitch analysis to estimate source
Or noise ⇒ use noise model for source
To model speech, where the vocal tract changes rapidly, LPC analyzes the sound frame-by-frame



VOSIM and Other Voice Synthesis Techniques

VOSIM

- uses a pulse train of $\sin^2$ pulses
- one formant (resonance) per VOSIM oscillator
- T gives formant position, $M + NT$ is period

FOF

- simulates the effect of a glottal pulse exciting a single formant resonance
- each FOF generator creates the response of a single formant to a single impulse (grain)
- FOF generators are allocated dynamically (since response to one impulse can overlap in time with response to the next impulse)

Vocoder / Phase vocoder

- decompose sounds using *filter banks*
- intended mainly for sounds with few partials
- estimate amplitude and phase of partial from successive frames of FFT
- reconstruct using IFFT

MacAulay-Quatieri (MQ) Synthesis

- identify and track sinusoids in STFT frames
- representation is list of trajectories of sinusoidal partials
- additive synthesis
- time stretching and frequency shifting are easy

- sinusoid track representation allows phase continuity
- noise is handled by using large numbers of sinusoids - not very efficient or flexible
- resonances not modeled
- no obvious cross-synthesis method

SMS – Spectral Modeling Synthesis

- extends MQ Analysis/Synthesis with explicit model for noise

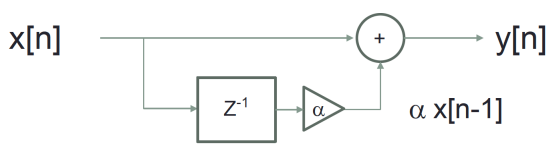
---

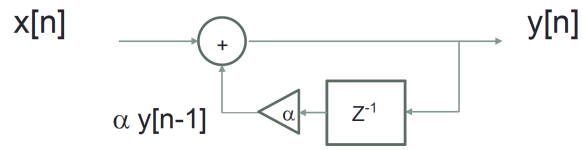
Chapter 11

Physical Models

Simple FIR (*Finite impulse response*) filter: `y[n] = x[n] + αx[n-1]` - delays are used along with some sort of averaging

Simple IIR (*Infinite impulse response*) filter: `y[n] = x[n] + αy[n-1]`





Chapter 11

Physical Models

Karplus-strong plucked string algorithm: perform table-lookup oscillator on noise and average adjacent samples ⇒ global exponential decay
Waveguide model: amplitude or flow is sum of lef-going and right-going waves

Lumped filters: model continuous distributed losses with discrete lump losses within waveguide or at the end
Non-linear elements are used to produce oscillation

Chapter 12

Spectral Interpolation Synthesis

Additive synthesis: every partial has independent frequency and amplitude but no simple parametric control
Table-lookup synthesis: relative amplitudes of all partials are locked in but frequencies are all harmonic
Spectral interpolation: interpolate between tables with coherent phases; restricted to harmonic spectra ⇒ reload tables with new spectra and do slow update
Problem: attacks are too rapid and inharmonic ⇒ use sufficiently long sampled attack, analyze phase of every partial and synthesize tables with matching phase

3D Sound

Cues to sense direction and distance:

- inter-aural time delay (direction in horizontal plane)
- spectral cues (elevation)
- reverberation (distance)
- spectral cues (amplitude, distance)

Head-related transfer functions (HRTF):

- duplex theory (interaural time difference (ITD) & interaural level difference (ILD, frequency dependent))
- measure effect of head on sound to each ear as a function of source direction then interpolate HRTF for sound motion

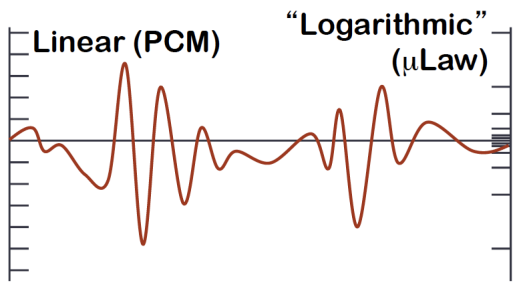
Room models: ray tracing
Doppler shift: distance adds delay that causes frequency shift
Panning: linear panning: sum of power $= A^2 + B^2 \Rightarrow$ "Hole in the Middle" effect
Constant power panning $\Rightarrow$ constant power

Chapter 13

Audio Data Compression

Coding redundancy: optimize coding of symbols; quantize in lossy coding; variable length coding

- PCM: linear encoding "uncompressed" audio
- μLaw (logarithmic encoding to 8 bits): closely spaced quantization level near 0

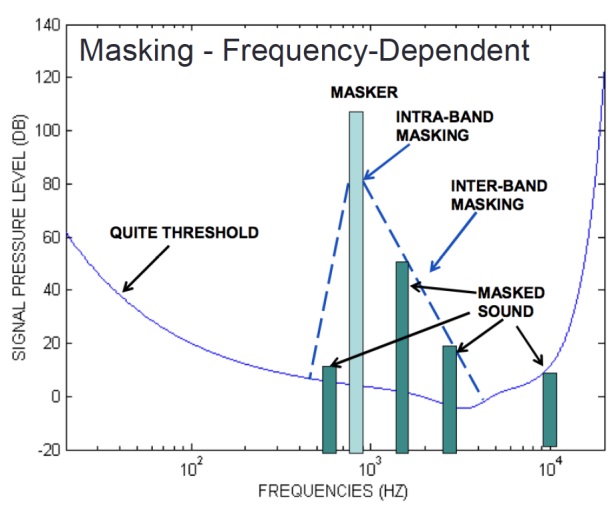


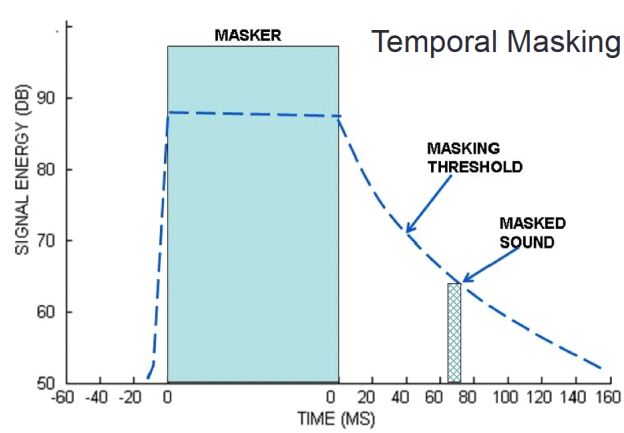
Intersample redundancy: represent runs of pixels; silence in audio; object description

- DPCM: encode sample differences $\Rightarrow$ 6dB improvement over PCM for low-quality speech
- ADPCM: adapt quantization size $\Rightarrow$ 10-11dB improvement over PCM for speech
- Adaptive prediction: nth order predictor forms weighted sum of past n samples

Psycho-Perceptual Coding and MP3

Psycho-perceptual redundancy: masking effects; perceptible information is important





- Loud sound in a channel masks soft sounds in adjacent channels
- Quantize as much as possible – using masking to hide quantization effects
- Represent sounds in each filter bank using lower sample rate
- Frequency domain: spectral data is more static and spectrum is sparse ⇒ efficient encoding

Quantization: reduce sample size ⇒ fewer bits ⇒ more quantization; acceptable when it is masked
Banded processing: shift the original spectrum by multiplying a sinusoid and get lower band ⇒ encode the signal as N bands at $1/N$ the sample rate

---

Chapter 14

Computer accompaniment: given a score, synchronize and synthesize an existing accompaniment part to a live solopart
Score alignment: given symbolic and audio files representing the same piece of music, find correspondingtime points between the two files
Style recognition: given a music audio sample, choose a style label that best characterizes the audio
Onset detection: segment music audio into notes
Human computer music performance: autonomous intelligent machine musicians